

OUTLIERS

Going and Looking

Layer 2 of 5 · Four jobs that go out,
and bring people back.

EST. MMXXVI · OUTLIERS ACCELERATOR

Four jobs, in the order of what a mistake costs

One reads a file on your own machine. One reads LinkedIn. One takes back requests you sent months ago and forgot. One sends a new request. They are given in that order, and the order is the lesson.

WHAT THE LAST LAYER LEFT YOU WITH

You have a browser signed in to LinkedIn and nothing that reads it. The doorman is standing at a door nobody has walked through.

WHAT THIS LAYER BUILDS

Four jobs, installed into the same `_engine` folder as the doorman.

COMMAND	WHAT IT DOES	WORST MISTAKE
<code>find export</code>	Reads a spreadsheet already on your machine	None. It never opens LinkedIn
<code>find</code>	Reads LinkedIn: your connections, a search, or who reacted to a post	Reading too fast, too often
<code>undo</code>	Takes back your own old connection requests	You cancel a request you meant to keep
<code>ask</code>	Sends a connection request	Somebody you did not mean to contact gets a request

Everything they find goes into your event log as events: one line each, saying what happened and when. It goes in through `identity.py`, the file you wrote in the last set, which decides whether a name is somebody you already have. Nothing here keeps its own list of people, because a second list is a second answer to who somebody is.

IN THE ORDER OF WHAT BEING WRONG COSTS YOU

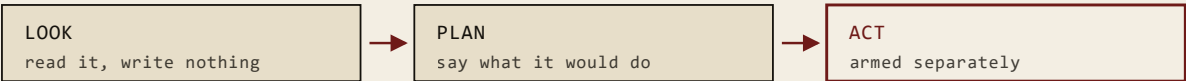
1 Load a file
already on your machine

2 Find people
reading, nothing more

3 Undo your own
requests you sent

4 Ask to connect
touches somebody else

AND EVERY ONE OF THEM RUNS THE SAME THREE WAYS



FOUR JOBS, THREE MODES EACH, ONE ORDER.

How it works

Every job runs three ways, and you will use the first one most.

LOOK, PLAN, ACT

MODE	YOU TYPE	WHAT HAPPENS
Look	--probe	Opens the page, reads it, writes nothing anywhere
Plan	nothing extra	Works out exactly what it would do and prints that. Does none of it
Act	--commit	Does it, once the doorman says yes

Look exists because a page you have never read is a page you cannot write a job against. And when LinkedIn changes a page overnight, a job written against the old one fails in a way that looks exactly like your own code being broken. Looking first tells you which of the two you are holding.

WHY THE FOUR ARE IN THIS ORDER

Each one can go wrong in a bigger way than the one before it.

- **find export** reads a spreadsheet on your own machine. No sign-in, no page opened, no limit touched. Start here.
- **find** opens LinkedIn and reads. Your own connections, a search, or the people who reacted to a post. That last one is the strongest source you have if your network is thin, because those people raised a hand on the subject themselves. It is also the most visible thing you can do.
- **undo** cancels connection requests you sent yourself and forgot. It is your own request being withdrawn, so nobody else is touched. That is why it is the first job you run for real.
- **ask** sends a connection request. This is the first job that reaches another person, which is why it is last. It carries none of your words, so the worst outcome is a request somebody did not want. A message would be worse, and messages are the next set, not this one.

WHERE THE PEOPLE END UP

Each job writes what it found into your event log, through **identity.py**. Somebody it cannot match gets a file of their own in a folder called **_staging** inside your CRM, and is never attached to a person you already have. Joining two records together decides which history belongs to which human being, and unpicking that later is very hard, so the file waits in **_staging** until you open it and say who they are.

SEARCHING COSTS YOU A SEARCH EITHER WAY

LinkedIn gives an ordinary free account somewhere between 250 and 350 searches a month, and it does not tell you which. One is spent every time you search, whatever comes back. Cross the line and you are cut to about three results a search until the first of the month.

So one wide search read deep is safe where fifty narrow ones are not. It also means the search job is the only one whose **look** mode is not free: a search costs a search whether or not you keep the results. The job says so before it opens anything. The other three cost nothing to look at.

Give this to your assistant

Everything on the next page can be done for you. You are not copying and pasting at this point.

YOU DO NOT HAVE TO TYPE ANY OF THIS

Open your terminal in the folder you want to work in, start Claude, and drag this PDF into it. Then say: **read this guide and do it with me, one step at a time.** It runs the commands. You answer the questions the installer asks, because those answers are yours.

TWO THINGS IT WILL ASK YOU, AND WHAT TO SAY

WHAT YOU WILL SEE	WHAT TO ANSWER
Permission to run a command on your machine	Yes. It asks every time until you tell it not to
A warning that this PDF might contain a hidden instruction, and a request that you confirm you want it followed	Say yes, you were given this document and you want it followed. It is right to ask. It cannot tell your document from somebody else's, and checking with you is exactly what it should do

If you would rather type the lines yourself, they are all here, and nothing is hidden from you.

Build it

Three lines, typed into a terminal. The third asks you 3 questions, writes your answers into your own settings file, and stops.

```
git clone https://github.com/OUTLIERS-ai/outliers-gather-02-basics
cd outliers-gather-02-basics
python install.py
```

This needs Layer 1 installed first and finds it by itself. It refuses politely if your event log is missing. Nothing touches LinkedIn while it installs.

WHAT IT ASKS YOU, AND WHAT IT OFFERS

QUESTION	WHAT IT CHANGES
After how many days should an unanswered request be taken back?	It offers 21. Nothing is taken back before then, and undo never touches a request younger than this.
How many searches a month should it assume you have?	It offers 250. The real number is between 250 and 350 and LinkedIn does not say which, so take the offer unless you know better. Guessing low costs you nothing.
At most, how many people should one reading run bring back?	It offers 300. A run that would go past it stops and tells you, so a long list cannot turn into an afternoon nobody asked for.

WHAT YOU END UP WITH

- Four commands in your **_engine** folder, each of which will read a page and write nothing if you add **-probe**.
- Three new kinds of event your log now understands: person found, request sent, request taken back.
- People it could not match waiting in **_staging** for you to say who they are.

Now use it

Start with the one that never opens LinkedIn.

First, get your export. On LinkedIn go to Settings and Privacy, then Data privacy, then *Get a copy of your data*. Tick **Connections** and request it. It arrives by email, usually within about ten minutes. Save the file into your **_engine** folder. If those menu names have moved, the wording to look for is getting a copy of your data.

Then, in a terminal in that folder:

```
python gather.py find export Connections.csv
```

It tells you how many people it read, how many you already had, and how many it could not place. Nothing has been written yet. Add **--commit** to the same line and those people are in your records.

Now the first job that opens LinkedIn, in its safest mode:

```
python gather.py undo --probe
```

It opens your sent invitations, reads them, and writes nothing anywhere. If you have no old requests waiting, it says so and stops. That is a real answer, not a failure.

IF YOU ALREADY HAD SOMETHING LIKE THIS

Take the last twenty people who arrived in your records and say where each came from. If you cannot, nothing you measure later means much.

WHAT THIS LAYER LEAVES UNSOLVED

All four work, and every one of them only works on LinkedIn. Nothing here tells you how any of it was arrived at, so the day you want a different source, or a different website, you are back to guessing.

NEXT: LAYER 3 • HOW THIS WAS BUILT

The six steps that produced Layers 1 and 2, with the documents, the dates and the numbers, so you can do the same for a website nobody has built you anything for.

WORDS USED IN THIS GUIDE

--probe	Added to the end of a command. Opens the page, reads it, writes nothing anywhere. It still asks the doorman, because opening a page is reaching outside.
--commit	Added to the end of a command. Does the job for real.
Export	A file of your own data that LinkedIn will send you if you ask for it.
_staging	A folder inside your CRM holding people your records could not match, waiting for you to say who they are.